

Building Interactive Dashboards

for Smarter Insights Using R Shiny

Mahesh Divakaran

Senior Tech Lead - R Developer,
Ephicity Life Science Analytics

Venkatesh Krishnan

Senior Advisor – Computation Statistics,
Eli Lilly & Company - India

Disclaimer

The data or the work is for presentation purposes only and does not involve any real patient data or violate any codes of confidentiality. The opinions expressed in this presentation and on the following slides are solely those of the presenters and not from the organization we belong to. It does not guarantee the accuracy or reliability of the information presented herein.

What you'll be able to do by 6:00 PM

Concepts

- Explain where R Shiny fits among clinical dashboard tools
- Describe the UI / Server / Reactive-engine model
- Recognize the building blocks of a clinical TFL dashboard

Hands-on

- Build and filter a mini TFL dashboard from ADaM data
- Export a table to HTML, Word and CSV, and a figure to PNG/SVG
- See a working pattern for LLM-assisted TFL generation

Setting Expectations: The Flow of Thought for Today

- 1. Why -- dashboards support recurring decisions, not one-off reports (this section)
- 2. What -- the UI / Server / Reactive-engine vocabulary every Shiny app is built from (Session 2)
- 3. See it -- watch that vocabulary become a real clinical scenario end-to-end (Session 3)
- 4. Build it -- put your hands on the same patterns in the Mini TFL Builder (Session 4)
- 5. Extend it -- see where AI/LLMs fit without breaking the guardrails you just learned to value (Session 5)
- Carry this thought through the day: every design choice you will see -- shared filtering, one renderer per table, visible N's -- is a direct answer to the 'science' and 'not a filter toy' ideas just covered

Five sessions, one working dashboard

Time	Topic	Detail	Who
09:45	Welcome & Introductions	Setting expectations for the day	IASCT EC member
10:00	S1 . Context & Use Cases	Why dashboards, where R Shiny fits	Mahesh, Venkatesh
10:45	S2 . Setup & Building Blocks	UI / Server / Reactivity / Modules	Mahesh, Venkatesh
11:35	S3 . Mini-Shiny Walkthrough	A clinical-trial scenario, live	Mahesh
13:30	S4 . Mini TFL Builder	Interactivity & output exports, hands-on	Venkatesh + Audience
15:45	S5 . AI & LLM Integration	Chatbot-driven TFL generation	Venkatesh, Mahesh
17:30	Closing & Concluding Remarks	Summary and open Q&A	IASCT EC member

Morning at a glance - concepts into a live build

Time	Topic	Detail	Who
09:45	Introduction of IASCT and speakers	Welcome note, expectations for the day	IASCT EC member
10:00	Dashboard purpose & clinical trial use cases	Static reports -> interactive analytics	Mahesh
10:15	R Shiny positioning	vs. Tableau, Spotfire, Qlik, SAS VA; network graphs, heatmaps	Venkatesh
10:30	High-level design considerations	Compliance, traceability, common mistakes	Mahesh
10:45	Environment setup	R, RStudio, packages, project structure	Mahesh
10:55	UI, server, reactivity, module concepts	Architecture overview	Mahesh
11:10	Building blocks for dashboard workflows	Data loading modules, reusable practices	Venkatesh
11:15	Interactivity in graphs	Dynamic, linked, drill-down visuals	Venkatesh

Afternoon at a glance - build, export, and go AI-assisted

Time	Topic	Detail	Who
11:35	Mini-Shiny Dashboard Walkthrough	Problem statement, table vs. figure, AE workflow	Speaker, Mahesh
12:15	Q&A and recap before lunch		All
13:30	Mini TFL Builder - hands-on build	Arm -> Population -> AE Summary Table	Speaker, Venkatesh + Audience
14:15	Interactivity for reviewing outputs	AE incidence viz, reactive filtering	Audience
15:00	Exporting to PDF and Word	Download/export functionality, reusability	Audience + Mahesh
15:45	AI/LLM integration patterns	With Ollama or OpenAI API	Venkatesh
16:15	Chatbot interface in R Shiny		Venkatesh
16:45	Illustrative use case with the mini TFL app		Venkatesh + Mahesh

What Is a Dashboard, and Why Does It Matter?

What it actually is

- Not a report with buttons -- a decision-support instrument built around a question someone must keep re-asking
- Compresses a large, changing dataset into a few signals a person can monitor at a glance, repeatedly
- The underlying question rarely has one answer -- it has a new answer every week of the trial

Why it matters here

- Shortens the distance between 'something changed in the data' and 'someone with authority noticed and acted'
- That distance is exactly where clinical and operational risk accumulates
- A validated app can replace dozens of one-off ad-hoc requests over a study's lifecycle

The Science Behind a Dashboard

Perception & cognition

- Pre-attentive attributes (color, position, size) are processed in under 250ms, before conscious reasoning
- Working memory holds about 4 plus/minus 1 chunks -- a dashboard's job is to reduce what a reviewer must hold in their head
- Gestalt grouping (proximity, alignment, similarity) is why a table reads as one statement, not a grid of numbers

Design consequence

- Data-ink ratio (Tufte): every pixel not carrying information competes with the pixels that are
- This is why 'no truncated axes', 'consistent color', 'visible N' are not decoration -- they are the mechanism
- Good defaults do the cognitive work up front, so the viewer's attention goes to the finding, not the layout

Why a Dashboard Is Not a Click-and-Filter Tool

The common misconception

- A dashboard's value is assumed to be its filters -- it is not; filters are the mechanism, not the point
- A pile of dropdowns with no guiding question just moves the analysis burden from programmer to reviewer
- That is a regression, not an improvement, if nothing else changes

What a good one does instead

- Encodes a point of view: which comparisons matter, which denominator is correct, which order tells the truth
- Filters let the viewer explore variations of a question that was already framed correctly
- Answers 'what should I be worried about right now', not just 'what can I look at'

Breaking the Traditional Mindset

Traditional (report-era) mindset

- 'The programmer produces the exact table specified; any new question means a new specification'
- Judgment about what matters is re-applied by hand on every ad-hoc request
- Interactivity bolted onto this thinking just produces a slower, more fragile static report

Dashboard mindset

- 'Validated logic is built once; the reviewer asks the next question themselves, inside guardrails already set'
- Rigor moves earlier -- into designing filters, denominators, and defaults -- rather than repeating per request
- What to unlearn: more clicks equals more power. What to adopt: the right default plus a few filters equals trust

01

SESSION 1 · CONTEXT & USE CASES

R Shiny in Clinical Trial Dashboards

Context, positioning, and design considerations for teams already comfortable with R Shiny basics.

Evolution from static reports to interactive analytics

What's changing

- Static PDF/RTF TFLs answer only the exact question they were built for
- Interactive dashboards let a reviewer ask the next question without a new programming request
- The underlying statistical logic doesn't change - only how a user reaches it does

Where it shows up

- Ad-hoc safety review during DSMB preparation
- Cross-functional study team check-ins
- Exploratory data review ahead of a formal analysis

Dashboards for study monitoring and decision support

Monitoring use cases

- Enrollment and disposition tracking across sites
- Ongoing AE signal monitoring outside the locked-database cycle
- Query and data-quality trend views for CRAs and DM

Decision support

- Faster DSMB / safety committee turnaround on ad-hoc questions
- Study teams self-serve filtered views instead of queueing programming requests
- One source of truth reduces duplicate one-off analyses

Who benefits, and how

Statisticians

- Explore subgroup and sensitivity views without re-running SAS/R batch jobs for every cut.

Programmers

- One validated app replaces dozens of one-off ad-hoc TFL requests over a study's lifecycle.

Clinicians & study teams

- Self-serve, guided filtering - no coding required to answer a review question.

Where R Shiny sits among clinical dashboard tools

Commercial BI platforms

- Tableau / Qlik - fast to configure, strong for general BI, licensed per seat
- Spotfire - established in life sciences, strong ad-hoc visual analytics
- SAS Visual Analytics - tight fit for SAS-based validated pipelines

R Shiny's niche

- Full R ecosystem access - the same language used to program ADaM/TLFs
- Open source: no per-seat licensing, fully customizable UI and statistical logic
- Best fit when the output logic itself - not just the visualization - is the hard part

Statistical graphics R Shiny renders natively

- Network graphsAdverse event co-occurrence, SOC/PT relationship maps, site-to-site enrollment flow - rendered with ggraph/visNetwork directly inside a reactive panel.
- Heatmaps & other statistical graphsLab shift heatmaps, correlation matrices, forest plots - all first-class ggplot2 geoms, styled once and reused everywhere in the app.

High-level design considerations for users familiar with R Shiny basics

Translating outputs into visuals

- A table row and a chart mark should represent the exact same underlying computation
- Preserve the N used for percentages visibly
 - don't let a filter silently change the denominator
- Keep clinical vocabulary (SOC, PT, TEAE) consistent between tables and figures

Compliance & traceability

- Every displayed number must trace back to a specific ADaM variable and filter
- Reproducibility: the same inputs must always produce the same output
- Log or version what population/treatment filters produced a given export

Common visualization mistakes to avoid

- Truncated or non-zero-based axes that visually exaggerate treatment differences
- Uncontrolled, non-consistent color mapping for treatment arms across tables and figures
- Omitting the N per arm, making percentages impossible to sanity-check
- Overplotting dense categorical data without faceting, sorting, or top-N filtering
- Interactive filters that silently change results with no visible indication a filter is active

SESSION 2 · SETUP & BUILDING BLOCKS

R Shiny Environment Setup and Dashboard Building Blocks

The architecture and vocabulary we'll reuse for the rest of the day, once, in the abstract.

Setting up the R Shiny development environment

Core setup

- R (4.x) + RStudio, or any IDE with an R language server
- Project-per-app structure: app.R or split ui.R/server.R
- renv recommended for locking package versions on a validated build

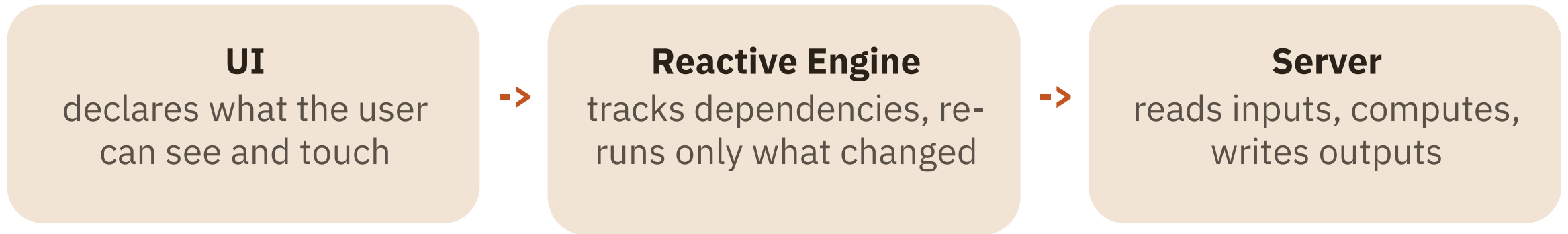
Recommended project layout

- One app.R for a self-contained app (what today's demo uses)
- A R/ folder for helper functions once the app grows past one file
- Keep data-loading, transformation, and rendering logic in separate functions from the start

The package stack behind today's demo app

- Shiny,bslib,dplyr / tidyr / purr, ggplot2
- gt, DT, haven, shinyWidgets
- pharmaverseadam, flextable + officer
- Each package owns one concern: shiny/bslib for app structure and theming, dplyr/tidyr for data shaping, gt/DT for tables, ggplot2 for figures, flextable/officer for Word export, and pharmaverseadam for realistic, ready-made ADaM sample data.

Architecture overview: UI / Server / Reactive Engine



Inputs, outputs, layouts and widgets

Inputs

- `selectInput`, `checkboxGroupInput` - dropdowns and multi-select
- `sliderInput` - numeric ranges, e.g. minimum AE incidence %
- `fileInput`, `dateRangeInput` - uploads and date pickers
- `shinyWidgets::switchInput` - a nicer boolean toggle

Outputs

- `tableOutput` / `gt_output` - tables
- `plotOutput` - static `ggplot2` figures
- `textOutput`, `uiOutput` - dynamic text and dynamically generated UI
- Every output has a matching `render*()` call in the server

Reactivity: the part that feels like magic

reactive()

- Wraps a computation that depends on inputs and caches its result
- Only re-runs when one of its dependencies actually changes
- Called like a function: `table_data()`, not referenced like a variable

observe() / observeEvent()

- Runs code for a side effect - not for its return value
- Used for things like showing a notification or updating another input
- `req()` pauses reactive execution silently until a condition is met

Building blocks for clinical trial dashboard workflows

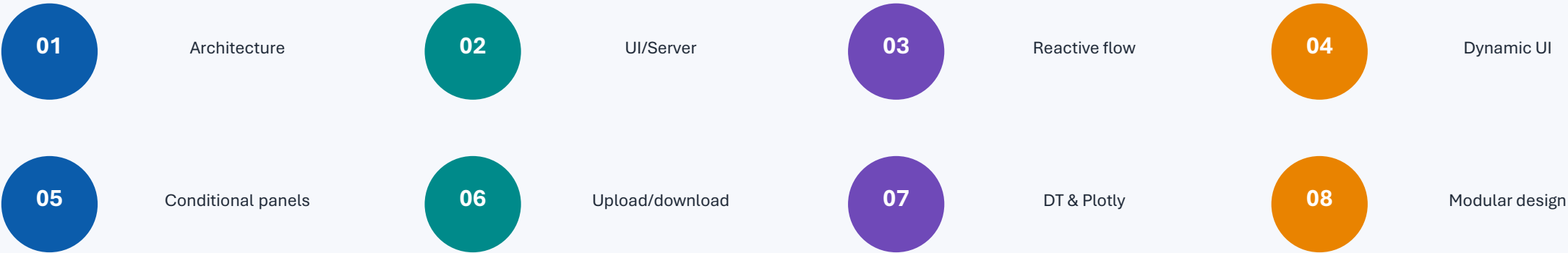
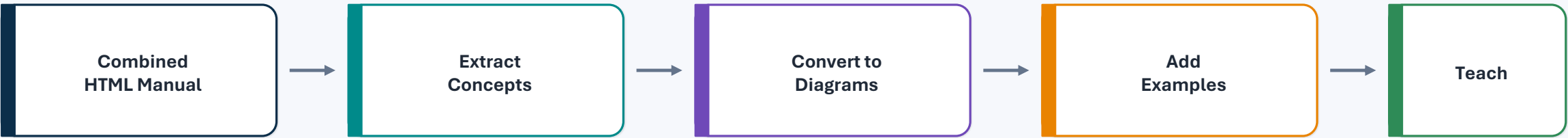
- Data loading modules - one reactive that resolves either built-in or uploaded ADaM data behind a single interface
- Reusable coding practices - shared filter/format helpers so every table and figure treats population flags and treatment arms identically
- R Shiny architecture (UI, Server, reactive logic) - keep each layer's responsibility narrow: UI never computes, Server never lays out

Interactivity in graphs in clinical trials

- Dynamic, linked, and drill-down visual interactivity for clinical review - from the external source developed by VK.
- Dynamic - a chart redraws immediately as filters change, no page reloadLinked
 - selecting a bar or point in one view filters another view on screen
- Drill-down - clicking a SOC-level bar reveals its underlying PT-level detail

Manual-to-slide transformation

The combined manual is converted into visual teaching blocks rather than text-heavy slides.



Visual learning roadmap

A single visual path that reflects the manual sections.



Instructor view: each module becomes a diagram slide, followed by a short runnable example or lab prompt.





R SHINY LEARNING DASHBOARD

Interactive Data Science with R

Select Theme


Learner

 Dashboard
 Data Explorer
 Visualizations
 Modeling
 Reports
 Deploy & Share
 Learn
 Help & Docs
 Settings

Learning Progress

75%

Level 3
Intermediate

[View Curriculum](#)


DATASETS LOADED
3
mtcars, iris, diamonds


VISUALIZATIONS
6
Charts & Plots


ACTIVE FILTERS
2
Species, Cylinders


RECORDS FILTERED
1.2K
of 3.5K total


LAST UPDATED
Just now
Automated Refresh

INPUT CONTROLS
[Reset](#)

Select Dataset
mtcars

X Axis
wt (Weight)

Y Axis
mpg (Miles per Gallon)

Color By (Optional)
cyl (Cylinders)

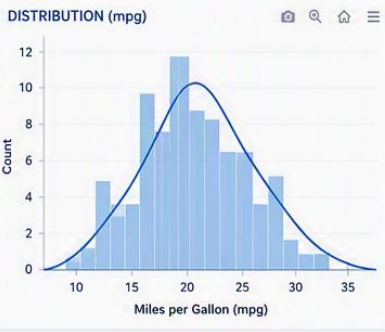
Point Size
Small Large

Filters
Cylinders
☒ 4 ☒ 6 ☒ 8
Transmission
☒ Automatic ☒ Manual
[Apply Filters](#)

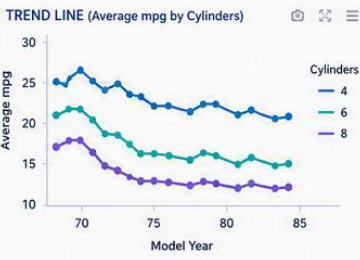
SCATTER PLOT



DISTRIBUTION (mpg)



TREND LINE (Average mpg by Cylinders)



DATA TABLE (Filtered Results)

mpg	cyl	dis	hp	drat	wt	am	gear	carb
22.8	4	108.0	93	3.85	2.32	1	4	1
24.4	4	146.7	62	3.69	2.20	0	4	2
32.4	4	78.7	66	4.08	2.20	1	4	1
30.4	4	75.7	52	4.93	1.62	1	4	2
33.9	4	71.1	65	4.22	1.84	1	4	1

Showing 1 to 5 of 11 entries

DEPLOY & SHARE



Deploy your app to the cloud
Share insights securely with your team or the wider organization.

- ☒ Connect to Posit Connect
- ☒ Publish Application
- ☒ Manage Access
- ☒ Monitor Usage

[Go to Deployment](#)

YOUR LEARNING JOURNEY


Level 1
R Fundamentals


Level 2
First Shiny App


Level 3
Intermediate Shiny


Level 4
Advanced Modular


Level 5
Enterprise Deployment

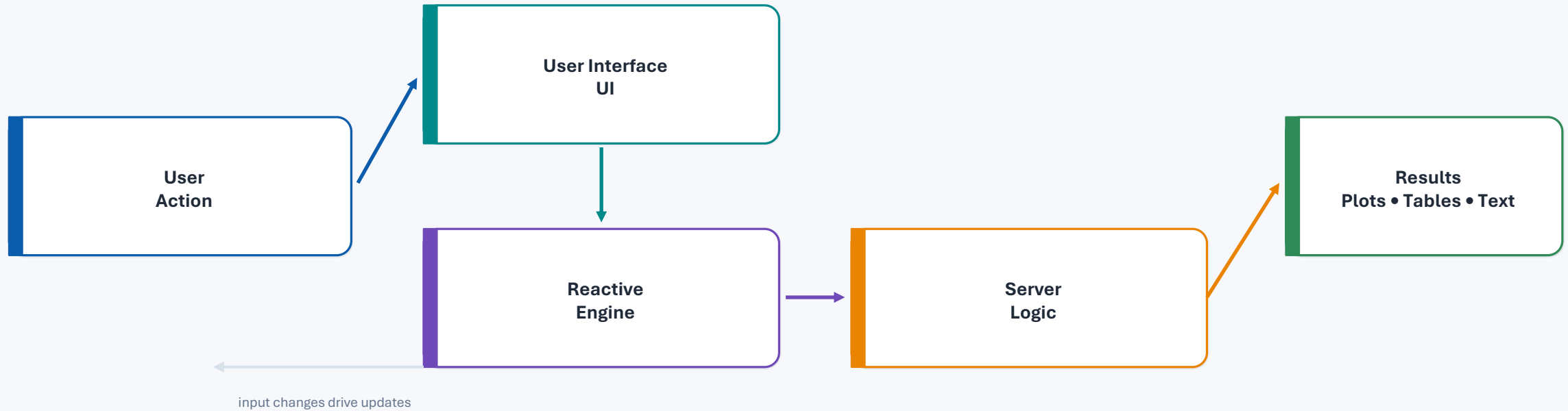


Keep going!
You're doing great.



Shiny architecture diagram

Manual concept: Shiny applications combine User Interface, Server Logic, and Reactive Engine.



Input controls as a visual palette

Manual concept: user inputs drive interactive applications.

textInput()
Text entry

selectInput()
Dropdown choice

sliderInput()
Numeric range

checkboxInput()
True/false

dateInput()
Date selector

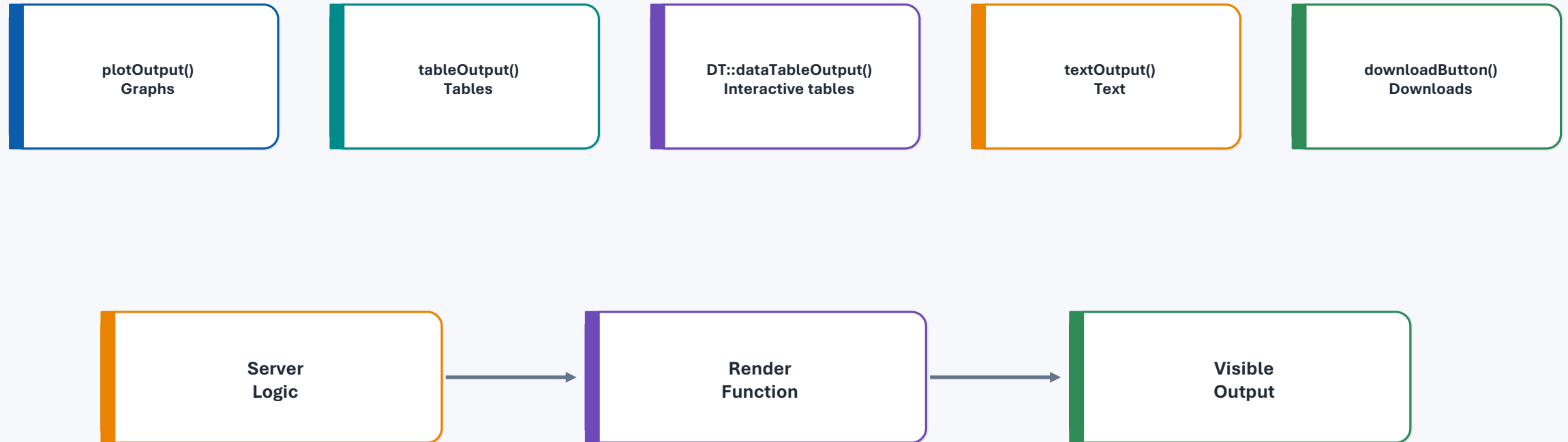
fileInput()
Upload files

Design principle: show only the input controls that support the learner exercise.



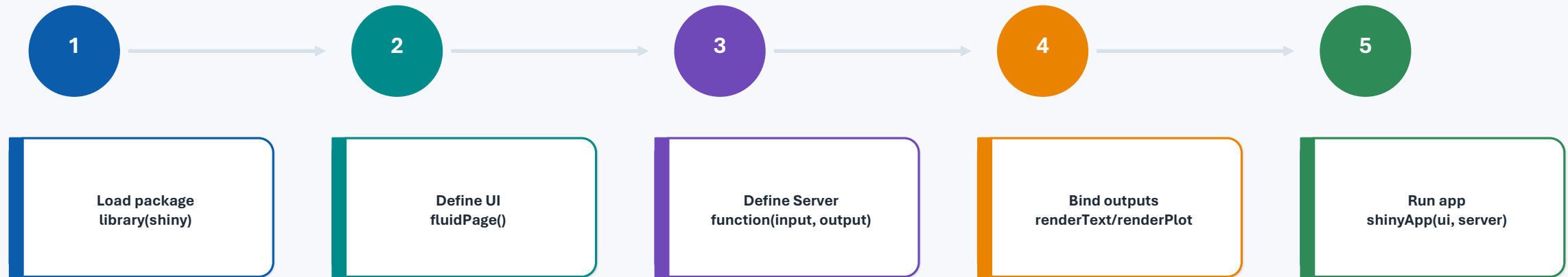
Output components as a dashboard palette

Manual concept: server logic renders tables, plots, text, interactive tables, and downloads.



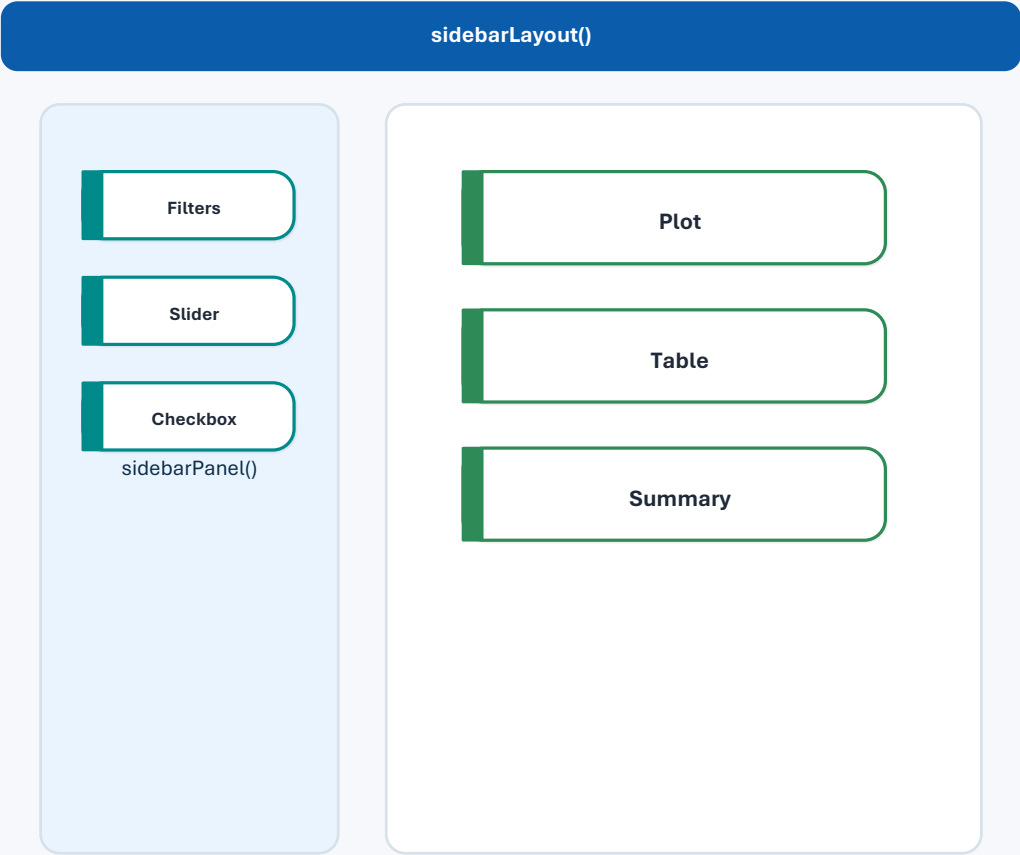
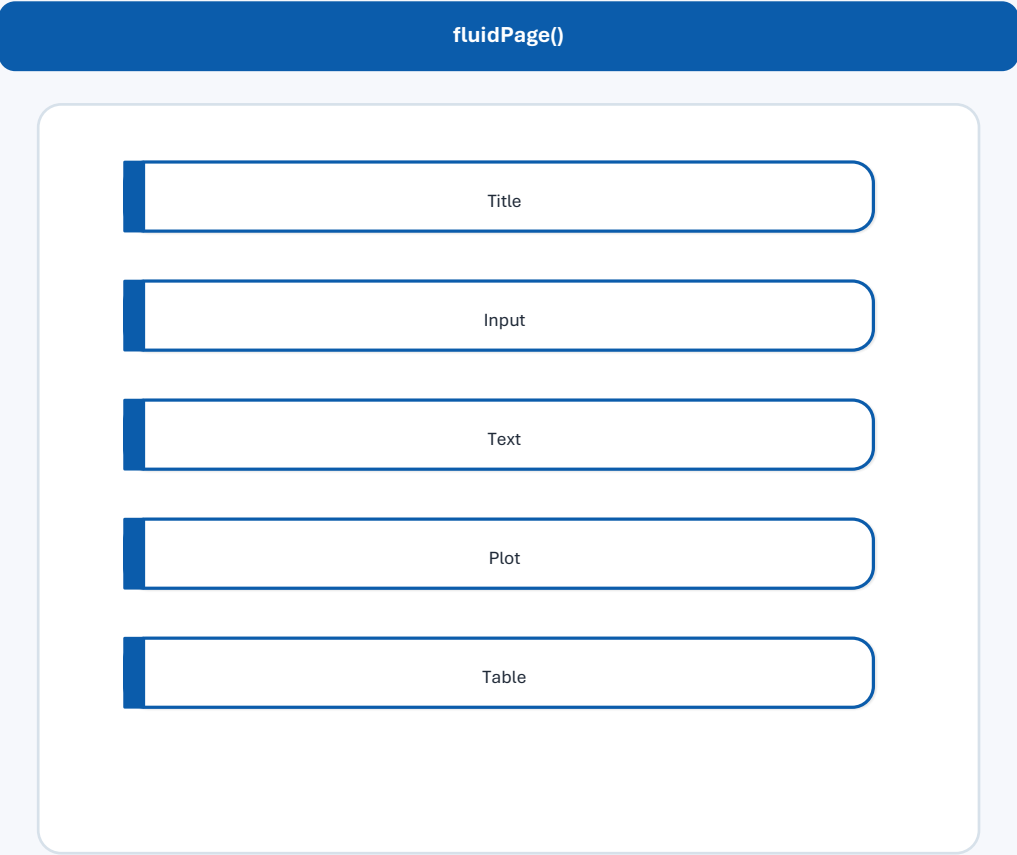
First Shiny app lifecycle

Manual concept: every app is built from UI, Server, and shinyApp().



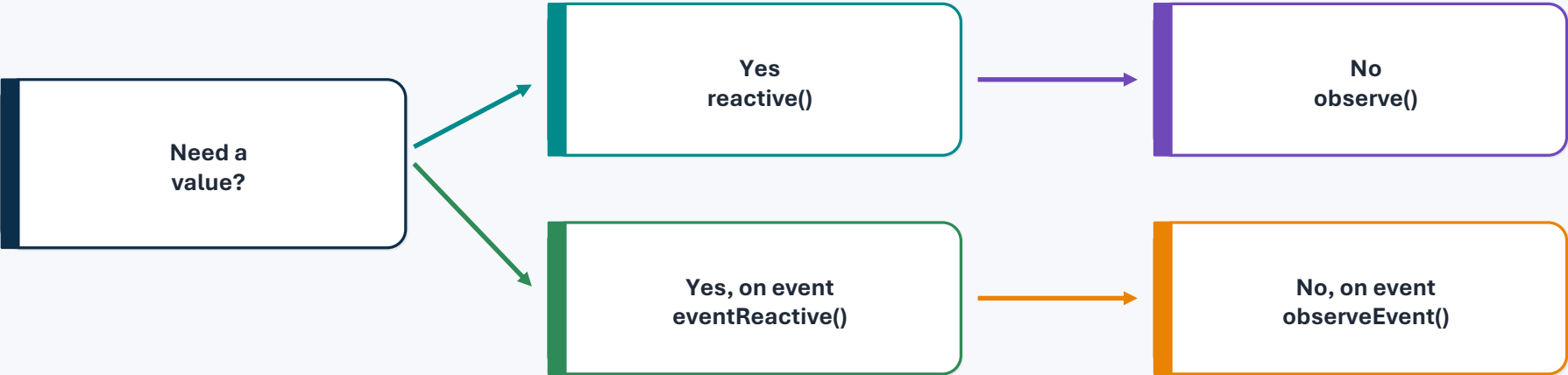
Layouts: fluidPage() vs sidebarLayout()

Manual concept: layouts organize inputs and outputs for usability.



Reactive functions decision map

Manual concept: choose a reactive tool based on whether it returns a value and what triggers it.



reactive(): Filtering, calculations, reusable logic

observe(): Notifications, logging, control updates

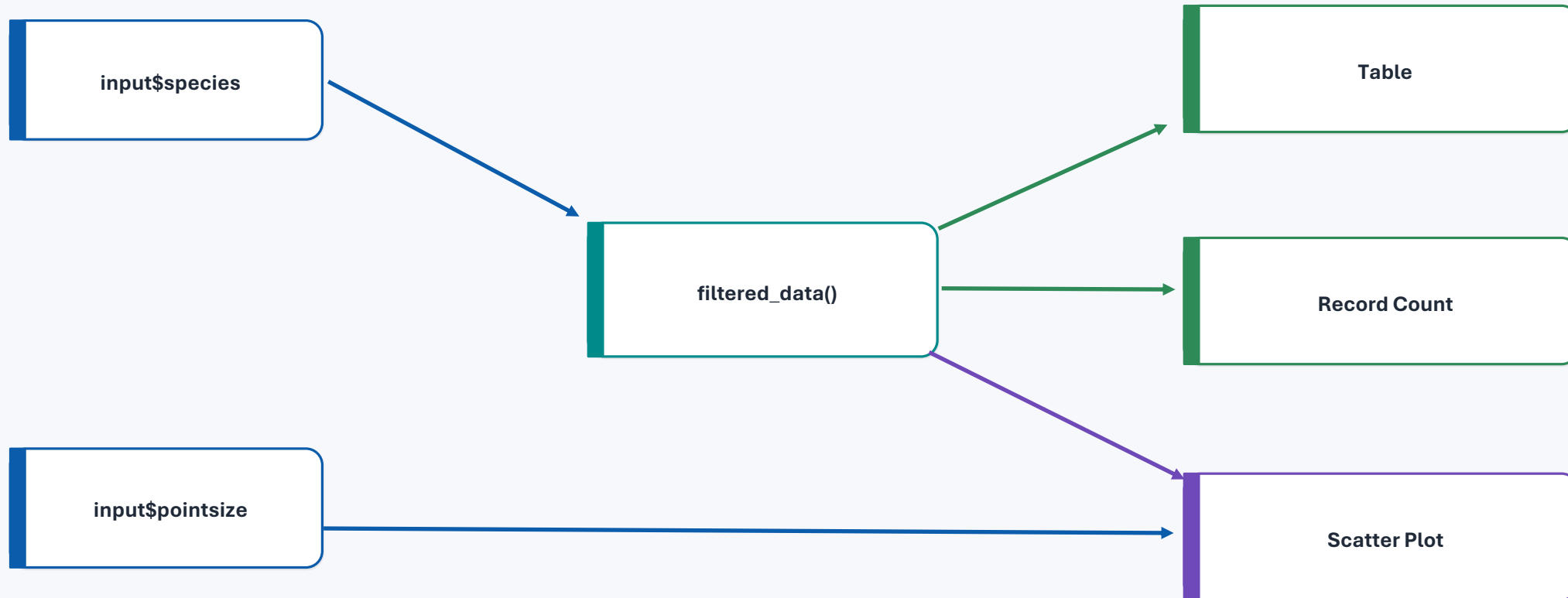
observeEvent(): Run, refresh, submit actions

eventReactive(): On-demand models or reports



Reactive dependency graph

Manual concept: reactive dependencies define how information flows between inputs, reactive expressions, and outputs.



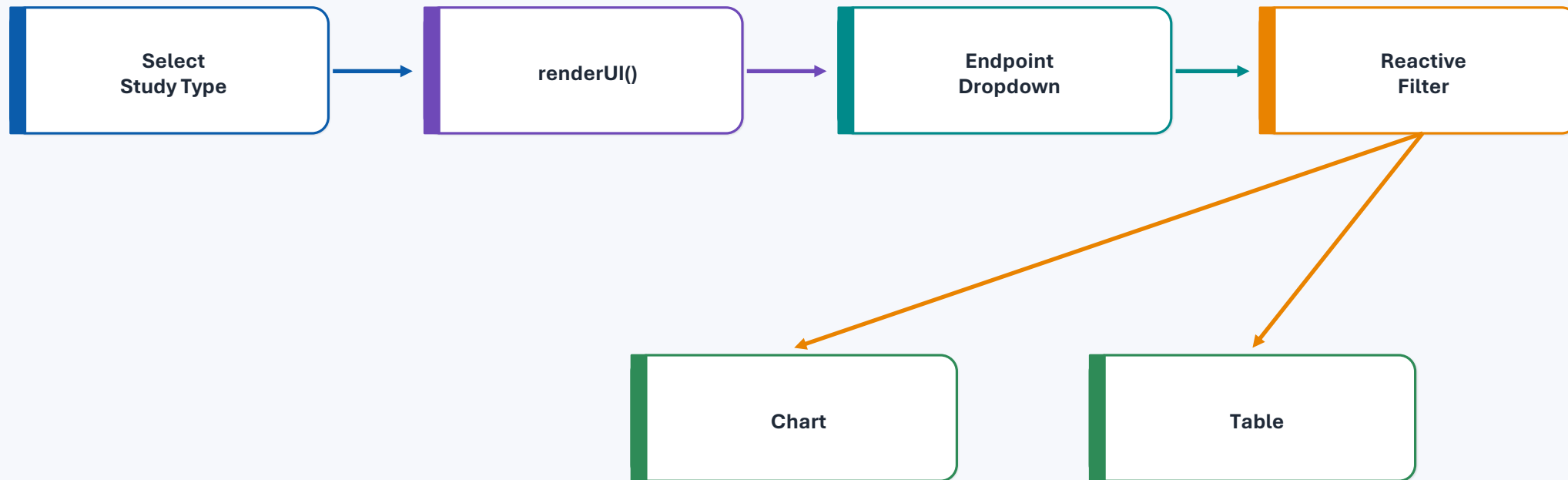
One shared reactive expression can feed multiple outputs, reducing repeated filtering logic.



Dynamic UI flow

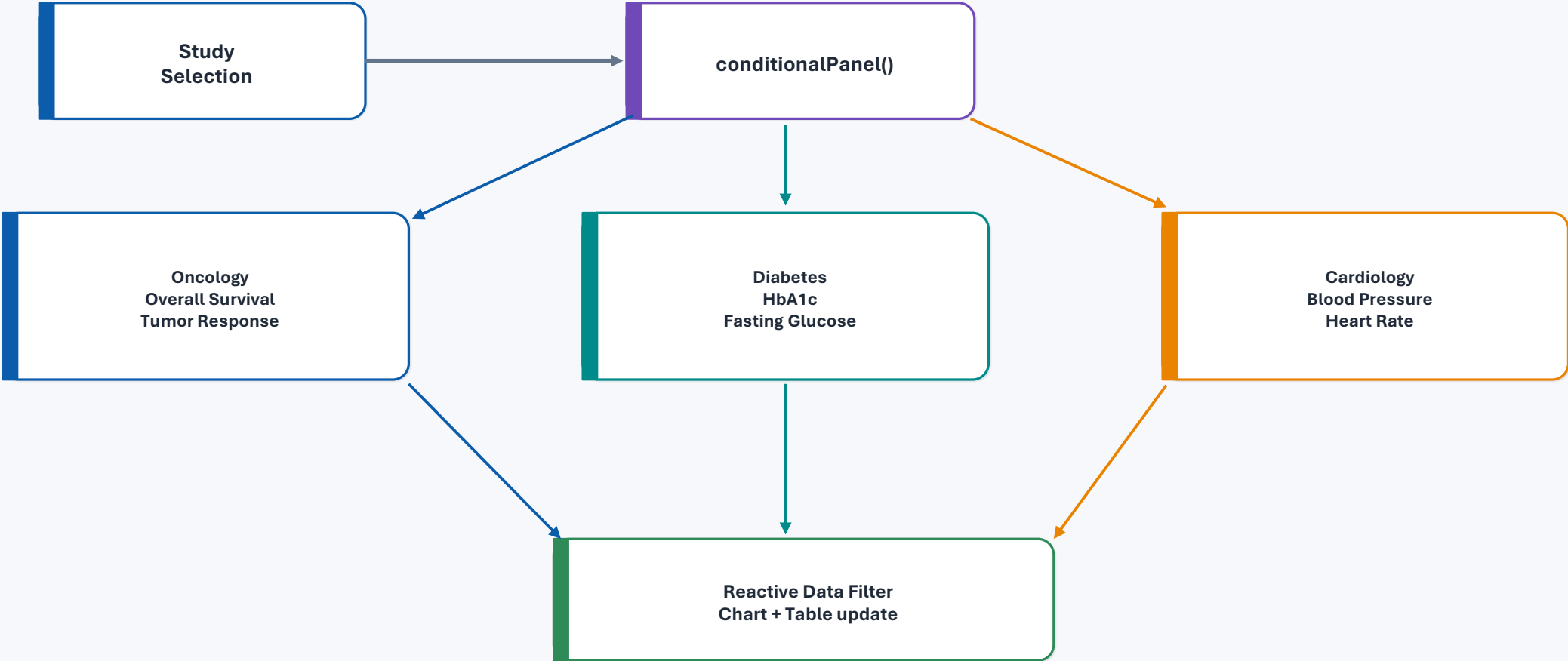
Manual concept: `uiOutput()` and `renderUI()` create controls based on user selections.

Example: Oncology -> survival/response endpoints | Diabetes -> HbA1c/glucose endpoints | Cardiology -> BP/heart-rate endpoints



Conditional panels workflow

Manual concept: conditionalPanel() shows or hides existing controls based on user choices.



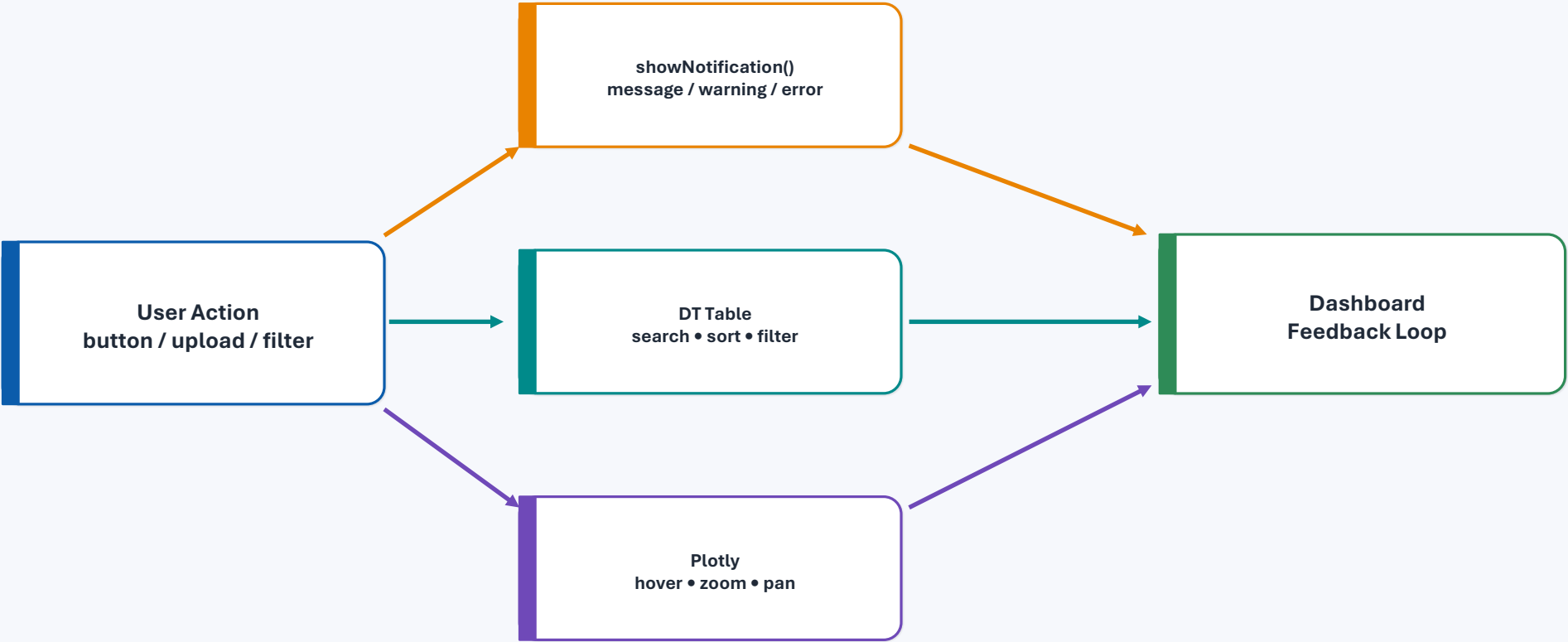
Workflow trio: tabs, upload/download, validation

Manual concept: production-ready apps organize content, accept files, export results, and validate inputs.



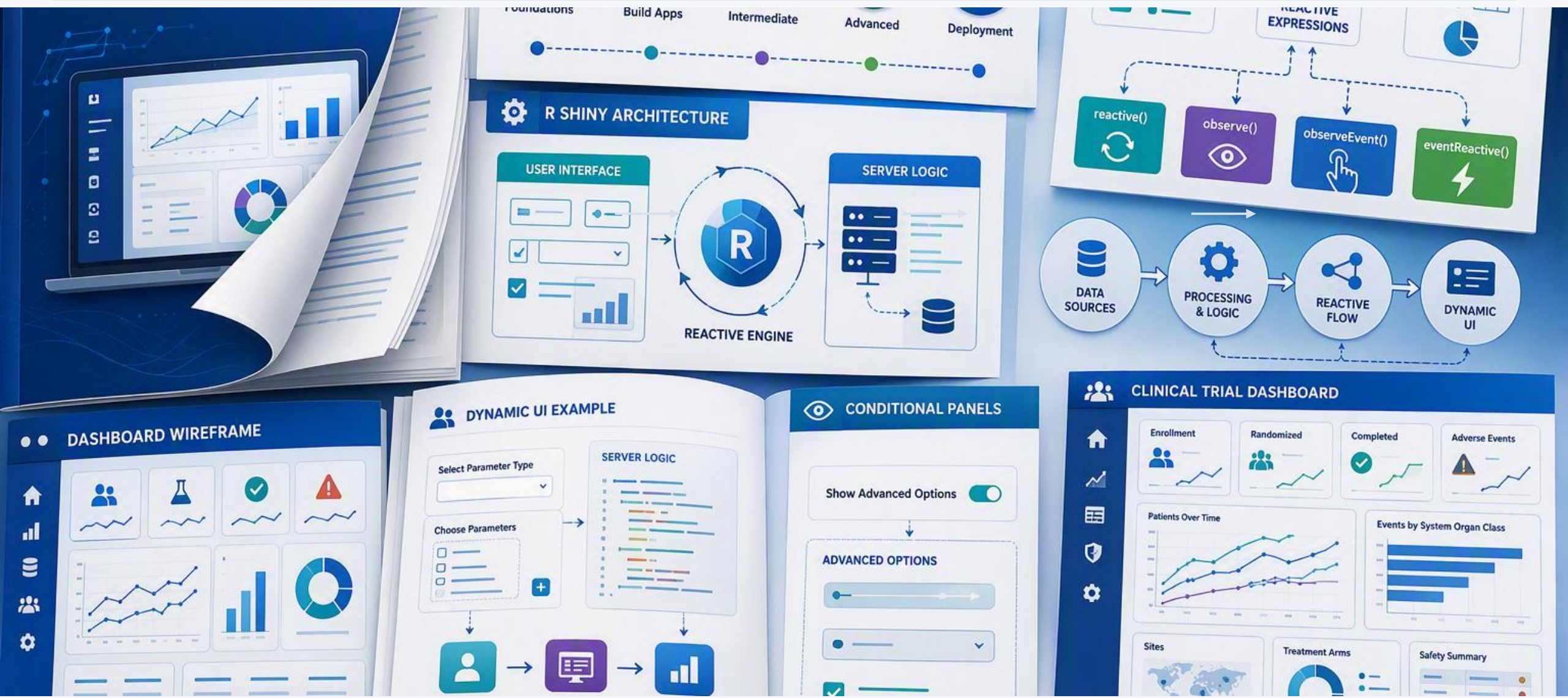
Interactive experience layer

Manual concept: notifications, DT tables, and Plotly improve usability and exploration.



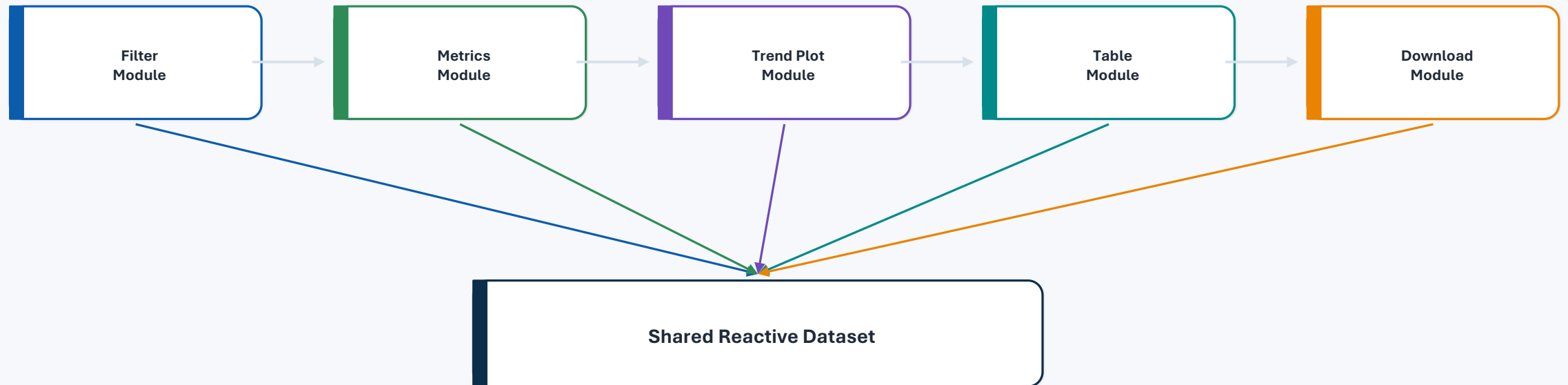
Level 4: modular architecture

Manual concept: split a large Shiny app into independent UI and server modules.



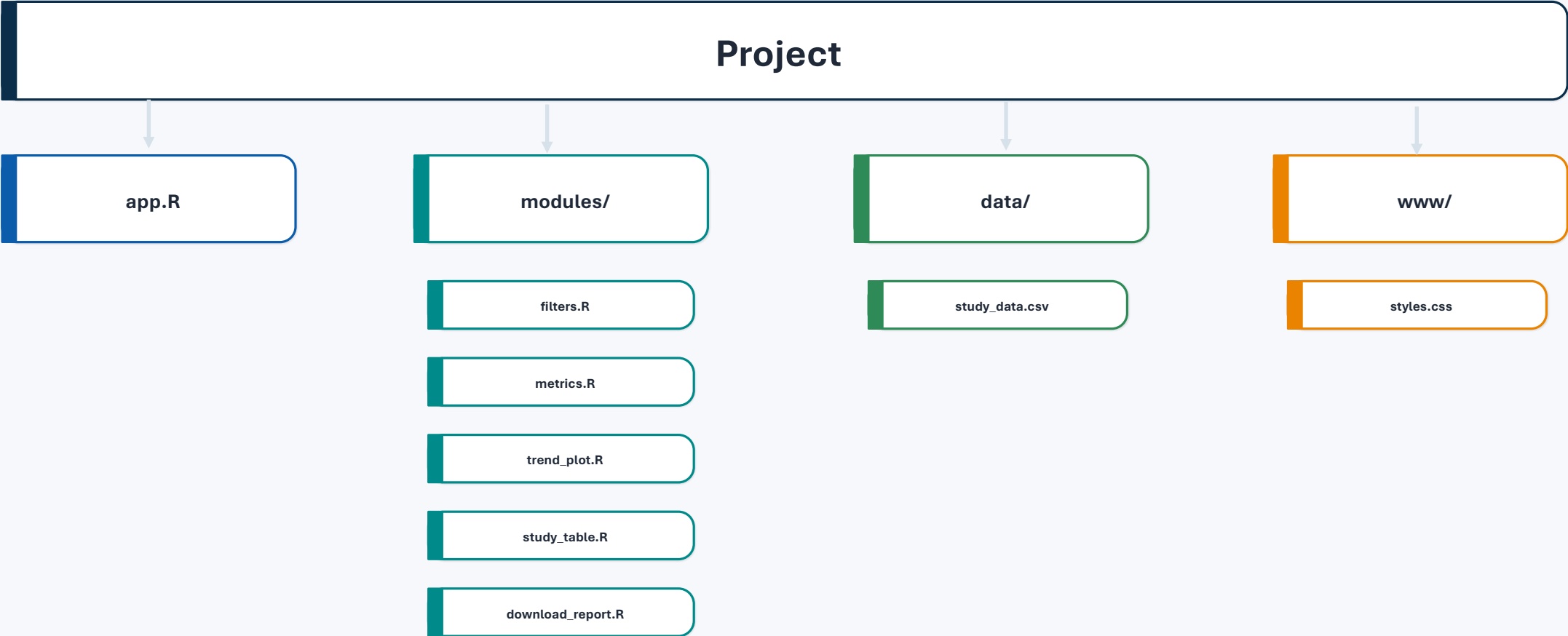
Level 4: modular architecture

Manual concept: split a large Shiny app into independent UI and server modules.



Project structure diagram

Manual concept: a production-style app separates modules, data, and static assets.



req() – the quiet guard used everywhere

- req(x) stops a reactive expression silently if x is missing, empty, or FALSE no error banner, the output simply doesn't render yet. It's how table_data() waits for treatment arms to be selected before attempting to build anything, and how datasets() waits for a file upload without crashing on NULL.

isolate() – reading a reactive without depending on it

- `isolate(x())` reads a reactive's current value without creating a dependency on it useful inside an `observeEvent()` when you want a button click to trigger the update, not every keystroke in an accompanying text field. Not used directly in today's app, but essential once a chat-style interface (Session 5) needs to read the current filter state only at send-time.

observeEvent() vs. reactive() – side effect vs. value

- A reactive() is called like a function and returns a value table_data() is read by multiple outputs. An observeEvent() runs for its side effect and returns nothing meaningful used for things like showNotification() or writing to a log. Mixing the two up is the most common source of confusing bugs for teams new to Shiny.

renderUI() / uiOutput() – server-generated UI

- Some UI can't be written once in the UI function because its choices depend on data that isn't known until runtime which population flags exist, which treatment arms are present. `uiOutput("tbl_pop_flag_ui")` in the UI pairs with `renderUI({...})` in the server to generate that control dynamically, every time the underlying dataset changes.

The `%||%` null-coalescing operator

- `a %||% b` returns `a` if it isn't `NULL`, otherwise `b`. Defined at the bottom of `app.R` and used in `table_data()` as `input$min_pct %||% 0` a one-line defensive default for a control that hasn't rendered on the very first page load yet.

SESSION 3 · MINI-SHINY WALKTHROUGH

Mini-Shiny Dashboard Walkthrough

A hands-on walkthrough for creating a mini-Shiny dashboard based on a clinical trial scenario - the bridge into today's live demo.

Problem statement and design decisions

The scenario

- A study team needs demographics, disposition, and AE summaries without filing a new programming ticket for every cut
- Requirement: one app, multiple TFL shells, consistent filtering across all of them
- Constraint: must work against real ADaM structures (ADSL, ADAE) so it generalizes beyond the demo

Design decisions

- One shared reactive data pipeline feeds every table and figure
- Population flag and treatment arms are selected per-tab, not globally, so Tables and Figures stay independent
- Every output owns its own export handler, but reuses the same underlying reactive

Table vs. figure generation; typical AE reporting workflow

Table generation

- Aggregate counts and percentages per treatment arm, formatted for direct clinical reading
- Structured with header rows, indentation (SOC -> PT), and a source note
- Exportable as-is to HTML/Word for a TLF shell

Typical AE workflow

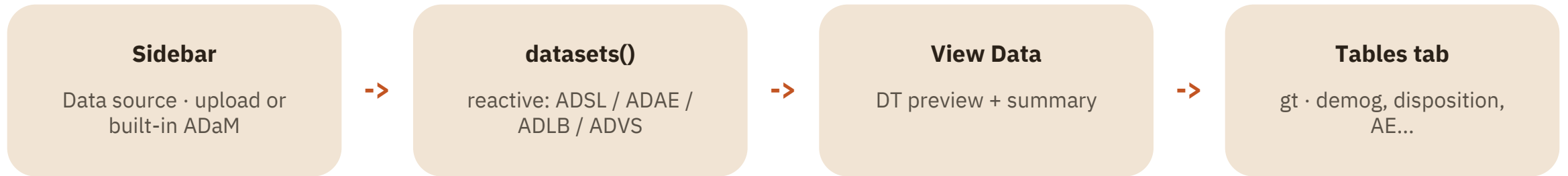
- Filter to the Safety Analysis Set (SAFFL == 'Y') and treatment-emergent events (TRTEMFL == 'Y')
- Summarize at Overview, then SOC/PT, then Severity level
- Visualize top incidence terms as a ranked bar chart for fast signal scanning

DEMO APP · ARCHITECTURE

Now Let's Open the App: A Mini TFL Builder

Built entirely in R Shiny on pharmaverseadam ADaM data. Everything from Session 2's concepts, applied.

Three tabs, one reactive data spine



What ships in the demo

5 Tables

- Demographics (14.1.1), Disposition (14.1.2), AE Overview (14.3.1), AE by SOC/PT (14.3.2), AE by Max Severity (14.3.3).

3 Figures

- Boxplot by Treatment (14.2.1), Histogram (14.2.2), AE Incidence Bar Chart (14.3.1).

3 Export formats

- HTML and CSV for tables, plus Word (.docx) via flextable/officer; PNG and SVG for figures.

Brand palette and treatment colors

```
BRAND_BLUE <- "#2C3E7A"  
TRT_PALETTE <- c("#546E7A", "#1565C0",  
"#B71C1C", "#2E7D32", "#6A1B9A",  
"#E65100", "#00695C", "#4527A0",  
"#37474F", "#880E4F")
```

- Defined once, at the top of the file, and referenced everywhere - the gt header, the sidebar title, and every ggplot2 figure
- 10 distinguishable colors support up to 10 treatment arms before any repeats
- Centralizing the palette means a rebrand is a one-line change, not a find-and-replace across every chart function

Table & figure menus, and their titles

```
TABLE_CHOICES <- c(
  "Demographics (T14.1.1)" = "demog",
  "Disposition (T14.1.2)" = "dispos",
  "AE Overview (T14.3.1)" = "ae_overview",
  "AE by SOC / PT (T14.3.2)" = "ae_socpt",
  "AE by Max Severity (T14.3.3)" = "ae_sev"
)
```

- The named vector is the dropdown label on the left, internal key on the right
- TABLE_TITLES maps the same keys to full TLF titles used in headers and exports
- Adding a 6th table means: one new entry here, one new build_*() function, one switch() case nothing else changes

Small formatting helpers, used everywhere

The three formatters

- `fmt_n_pct(n, N) -> "12 (34.3%)"`
- `fmt_mean_sd(x) -> "45.2 (12.1)"`
- `fmt_med_rng(x) -> "44.0 [21.0, 78.0]"`

Why it matters

- Every table in the app calls these formatting is consistent by construction, not by convention
- A single decimal-place change (e.g. regulatory requirement update) touches 3 lines total
- Each formatter is a pure function: same input always produces the same string, easy to unit test

shiny + bslib – the app's skeleton

shiny

- Core reactivity engine, input/output bindings, downloadHandler(). Everything else in this app is decoration on top of shiny's dependency graph.

bslib

- Bootstrap 5 theming and modern layout primitives `page_sidebar()`, `navset_tab()`, `card()`, `layout_columns()` used throughout the UI.

shinyWidgets

- Fills gaps in base shiny's input set `switchInput()` gives the Show Total toggle a cleaner look than a checkbox.

dplyr + tidyr + purrr – the data-shaping trio

dplyr

- `filter()`, `mutate()`, `count()`, `group_by()`, `summarise()` every aggregation in every `build_*`() function is dplyr verbs chained with `|>`.

tidyr

- `pivot_wider()` and `pivot_longer()` turn long counted data into the wide, one-column-per-arm shape a clinical table needs.

purrr

- `map_dfr()` drives the SOC/PT nesting in `build_ae_socpt()` and the row-by-row TEAE category logic in `build_ae_overview()`.

gt + DT – two table packages, two jobs

gt

- Publication-quality, styled clinical tables headers, spanners, conditional cell styling, source notes. This is what every Tables-tab output uses.

DT

- Interactive, sortable, filterable data grids used only in the View Data tab, where the goal is raw exploration, not a formatted TLF.

Why both

- gt for the deliverable, DT for the working view using one package for both would compromise either the polish or the interactivity.

ggplot2 – every figure, one grammar

Layered grammar

- `geom_boxplot()`, `geom_histogram()`, `geom_col()` each figure builder composes geoms onto one `ggplot()` call, the same mental model every time.

tfl_theme

- A single theme object applied to all three figures keeps typography, gridlines, and legend placement consistent app-wide.

Dual-purpose output

- The same plot object renders on screen via `renderPlot()` and exports via `ggsave()` at print resolution no separate export-only code path.

haven + pharmaverseadam + flextable/officer

haven

- Reads real SAS transport-style .sas7bdat files directly into R critical for a tool meant to work against actual clinical deliverables.

pharmaverseadam

- Ships realistic, ready-made ADSL/ADAE/ADLB/ADVS sample data the demo works out of the box with zero setup.

flextable + officer

- Rebuilds any gt table as an editable, submission-style Word table
officer::read_docx()
assembles the .docx entirely in memory.

How the packages divide responsibility

- Notice the pattern: each package owns exactly one concern, and no two packages compete for the same job. dplyr/tidyr never touch presentation; gt/ggplot2 never touch aggregation. That separation is what makes every `build_*` function testable on its own, independent of Shiny running at all.

prep_adsl() – part 1: population & treatment filter

```
prep_adsl  
mutate(TRT = factor(.data[[trt_var]],  
levels = trts))
```

- Every table and figure builder calls prep_adsl() first it's the single choke point for filtering logic
- .data[[pop_flag]] uses tidy-eval to filter by a column name stored in a variable essential since the population flag is a user selection, not hardcoded
- factor(..., levels = trts) locks treatment display order to the order the user selected arms in

prep_adsl() – part 2: synthesizing the Total column

```
N <- setNames(as.integer(table(d$TRT)), trts)
all_lvls <- if (show_total) c(trts, "Total")
else trts
if (show_total) {
  d <- bind_rows(d, mutate(d,
    TRT = factor("Total", levels = all_lvls)))
  N <- c(N, Total = sum(N))
}
list(d = d, N = N, all_lvls = all_lvls)
}
```

- The "Total" column isn't special-cased downstream it's manufactured by duplicating every row with TRT relabeled to "Total"
- Because it's just another factor level, every count()/pivot_wider() call downstream treats it identically to a real arm
- Returns a 3-part list d, N, all_lvls that every builder deconstructs the same way

wide_counts() – count, join, pivot in one call

```
wide_counts
count(TRT, across(all_of(grp_cols))) |>
right_join(expand.grid(...), # fills implicit
zeros
by = c("TRT", grp_cols)) |>
mutate(n = replace_na(n, 0L),
val = fmt_n_pct(n, N[as.character(TRT)])) |>
pivot_wider(names_from = TRT, values_from = val)
}
```

- `expand.grid() + right_join()` is the trick that turns "0 subjects had this category" from a missing row into an explicit "0 (0.0%)"
- Reused by `build_demog()` (sex, race), `build_dispos()` (status), and `build_ae_sev()` (severity) one function, three tables
- Ends in wide format: one column per treatment arm, ready to hand to `gt()`

hdr_row() – the \"(N=xx)\" header row

A tiny helper that builds a single-row tibble like Characteristic | (N=64) | (N=62) | (N=126) using the treatment N's computed in prep_adsl(). Every table prepends one of these via bind_rows() so the reader always sees the denominator before reading a single percentage directly addressing the \"don't hide the N\" design principle from Session 1.

build_demog() – Demographics (T14.1.1)

```
age_tbl group_by(TRT) |>
  summarise(\Mean (SD)\ = fmt_mean_sd(AGE),
    \Median [Min, Max]\ = fmt_med_rng(AGE)) |>
  pivot_longer(-TRT) |>
  pivot_wider(names_from=TRT)
cat_blk
mutate(Variable = label, .before = 1)
```

- Age is summarized as continuous (mean/SD, median/range); sex and race are summarized as categorical via wide_counts()
- SEX is relabeled Female/Male and RACE is title-cased before counting, so display formatting never leaks into the aggregation logic
- Final table is a bind_rows() stack: header row, age block, sex block, race block

build_dispos() – Disposition (T14.1.2)

```
status
mutate(Category = c("Completed", "Discontinued")[
  match(Category, c("COMPLETED", "DISCONTINUED"))])
|>
replace_na(list(Category = "Other"))
```

- Defensive check first: if EOSSTT isn't in the uploaded data, the function returns a graceful "data unavailable" placeholder table instead of erroring the whole app
- Reuses wide_counts() again the third table to lean on the same shared helper
- Raw ADaM codes (COMPLETED) are relabeled to sentence case for clinical readability at render time only

build_ae_overview() – TEAE Overview (T14.3.1)

```
rows <- list(  
  list("Any TEAE", quote(TRUE)),  
  list("Any Serious TEAE", quote(AESER=="Y")),  
  list("TEAE Leading to Death",  
    quote(!is.na(AESDTH)&AESDTH=="Y")),  
  ...  
)  
result <- map_dfr(rows, ~cnt(base_ae, .x[[2]]))
```

- Each row is a quote()d condition unevaluated R code stored as data, then evaluated later inside cnt() via filter(!cond_expr)
- This pattern makes adding a new TEAE category (e.g. "Leading to Dose Reduction") a one-line addition to the rows list, no new function needed
- tryCatch() around cnt() means a missing variable degrades to "N/A" per-row rather than crashing the table

build_ae_socpt() – AE by SOC/PT (T14.3.2), part 1

```
mk_counts
distinct(USUBJID, TRT, !!!syms(id_cols)) |>
count(TRT, !!!syms(id_cols)) |>
mutate(pct = 100 * n / N[as.character(TRT)]) |>
pivot_wider(id_cols=id_cols, names_from=TRT,
values_from=c(val,pct))
}
soc_df <- mk_counts("AEBODSYS")
pt_df <- mk_counts(c("AEBODSYS","AEDECOD"))
```

- distinct(USUBJID, ...) before counting is the key correctness step: a subject with 3 events of the same PT is still counted once for incidence
- One helper, mk_counts(), generates both the SOC-level and PT-level count tables by just changing which grouping columns are passed in
- !!!syms(id_cols) is tidy-eval splicing a character vector of column names into dplyr verbs that normally expect bare column names

build_ae_socpt() – AE by SOC/PT (T14.3.2), part 2

```
rows
mutate(Level = "SOC")
pt_rows
arrange(desc(pct)) |>
mutate(Term = paste0(" ", Term),
Level = "PT")
bind_rows(soc_row, pt_rows)
})
```

- SOC rows are sorted by mean incidence across arms; within each SOC block, PT rows are sorted by their own descending incidence a nested sort, built with two `arrange()` calls at two levels of `map_dfr()`
- PT rows get 3 leading spaces prepended to Term for visual indentation a simple text trick that avoids a more complex nested-table rendering approach
- Level ("SOC" / "PT") travels through to the gt renderer, which uses it purely for styling (bold SOC rows), then is dropped before export

build_ae_sev() – AE by Max Severity (T14.3.3)

```
sev_lvls  
distinct(USUBJID, TRT, AESEV),  
"AESEV", N, lvls  
) |> rename(Severity = AESEV)
```

- The 4th call site for wide_counts() in the app the same helper now handles a third categorical variable with zero new counting logic
- distinct(USUBJID, TRT, AESEV) again ensures each subject is counted once per severity level, not once per event
- Explicit factor levels ordering keeps Mild -> Moderate -> Severe in clinical reading order regardless of alphabetical sort

to_gt() – structure and spanners

```
gt_tbl  
select(-any_of("Level")) |>  
gt(rowname_col = first_col) |>  
tab_header(title = md(paste0("***",title,"***"))  
|>  
tab_spanner(label = md("**Treatment Group**"),  
columns = all_of(active_trt))
```

- Every one of the 5 tables passes through this single renderer table-specific logic lives in the build_*() functions, presentation logic lives here, once
- Level (used only for internal styling decisions) is dropped from the visible output right before rendering
- The "Treatment Group" spanner groups arm columns under one header, visually separating them from the row-label column

to_gt() – conditional styling

```
if (tbl_type == "ae_socpt" && "Level" %in%
names(df)) {
  soc_rows tab_style(
    style = cell_text(weight = "bold"),
    locations = cells_body(rows = soc_rows, columns
= first_col))
}
if (has_total) gt_tbl
tab_style(style = cell_fill(color="#F0F4FF"),
locations = cells_body(columns="Total"))
```

- Row-level bolding for SOC rows is conditional on both the table type and the presence of the Level column a defensive double-check
- The Total column, when shown, gets a distinct fill color and bold weight in both the body and the header, so it visually reads as a summary column, not just another arm
- Closes with tab_source_note() stating the analysis set and percentage basis required context for a clinical reader on every export

tfl_theme – one look for every chart

```
tfl_theme <- theme_classic(base_size = 13) +
  theme(
    plot.title = element_text(face="bold", size=13),
    plot.subtitle = element_text(color="grey40"),
    legend.position = "bottom",
    strip.background = element_rect(fill="#E3EBF6"),
    panel.grid.major.y =
      element_line(color="grey92")
  )
```

- Added to every ggplot object with + tfl_theme one theme object, reused across boxplot, histogram, and AE bar chart
- Only horizontal gridlines are kept (panel.grid.major.y) a deliberate choice to reduce chartjunk while keeping value-reading easy
- Legend fixed at the bottom keeps every figure's layout predictable when embedded in a report or slide

build_boxplot() – Distribution by Treatment (F14.2.1)

```
pl <- ggplot(d, aes(TRT, .data[[yvar]]),  
  fill=TRT))  
if (show_violin) pl <- pl +  
  geom_violin(alpha=0.2)  
pl <- pl + geom_boxplot(alpha=0.7,  
  outlier.shape = if(show_pts) NA else 19)  
if (show_pts) pl <- pl +  
  geom_jitter(aes(color=TRT),  
    width=0.12, alpha=0.5)
```

- Layers are added conditionally based on live user toggles `show_violin` and `show_pts` by mutating the same `pl` object in place before it's returned
- When jittered points are shown, boxplot outliers are hidden (`outlier.shape = NA`) to avoid double-plotting the same subjects
- N per arm is baked directly into the x-axis labels: "Placebo\\n(n=64)" the same "never hide the N" principle from Session 1, applied to a figure

build_histogram() – Continuous Variable Distribution (F14.2.2)

```
if (facet_trt) {  
  pl <- pl + geom_histogram(bins=bins) +  
  facet_wrap(vars(TRT), ncol=1)  
} else {  
  pl <- pl + geom_histogram(bins=bins,  
    position="identity", alpha=0.55)  
}  
if (show_density) pl <- pl + geom_density(  
  aes(y=after_stat(count*diff(range)/bins),  
    color=TRT))
```

- Two display modes toggle the whole chart structure: faceted-by-treatment (stacked panels) vs. overlaid with transparency same data, different comparison strategy
- `after_stat(count * diff(range)/bins)` rescales the density curve's y-axis to match the histogram's count axis, so overlay actually lines up visually
- Bin count is a live sliderInput a direct, visible lesson in how binning choice changes a distribution's apparent shape

build_ae_bar() – Top TEAEs by Incidence (F14.3.1)

```
adae |> filter(TRTEMFL=="Y") |>
distinct(USUBJID, TRT, AEDECOD) |>
count(TRT, AEDECOD) |>
mutate(pct = 100*n/N[as.character(TRT)]) |>
group_by(AEDECOD) |> mutate(max_pct=max(pct)) |>
filter(AEDECOD %in% slice_max(...,
n=top_n)$AEDECOD) |>
mutate(AEDECOD = fct_reorder(AEDECOD, max_pct))
|>
ggplot(aes(pct, AEDECOD, fill=TRT)) +
geom_col(position=position_dodge(0.7))
```

- "Top N" ranking uses each term's maximum incidence across any single arm a term dominant in only one arm still surfaces, which matters for safety signal review
- fct_reorder() sorts the y-axis by that ranking so the chart reads top-to-bottom as highest-to-lowest incidence, standard practice for incidence bar charts
- top_n is a live slider (5-25 terms) the reviewer controls signal-to-noise directly

page_sidebar() + bslib theming

```
ui <- page_sidebar(  
  title = tags$span(tags$b("TFL"), " Builder"),  
  theme = bs_theme(  
    bootswatch = "flatly", primary = BRAND_BLUE,  
    font_scale = 0.95, "navbar-bg" = BRAND_BLUE  
  ),  
  sidebar = sidebar(width = 290, ...),  
  navset_tab(id = "main_tabs", ...)  
)
```

- `bslib::bs_theme()` gives full Bootstrap 5 theming from four arguments no custom CSS file needed for brand colors
- `page_sidebar()` is a bslib layout primitive: a persistent left sidebar plus a main content area, which is exactly the global-controls / per-tab-content split this app needs
- The sidebar carries only global concerns (data source); everything analysis-specific lives inside each tab, one layer down

Data source: built-in vs. upload

```
radioButtons("data_source", NULL,  
choices = c("pharmaverseadam (built-  
in)"="builtin",  
"Upload my own data files"="upload"))  
conditionalPanel("input.data_source ==  
'upload'",  
fileInput("upload_files", ...,  
accept = c(".sas7bdat", ".csv", ".rds")))
```

- conditionalPanel() uses a JavaScript expression string, not R it toggles visibility client-side with zero server round-trip, so the UI feels instant
- Accepting .sas7bdat directly (via haven) means a study team can point the app at real SAS transport-style datasets, not just CSV exports
- Files are matched to ADSL/ADAE/etc. by filename, not by asking the user to manually label each upload

Split layout: controls left, output right

```
layout_columns(col_widths = c(3,9),
div(style="overflow-y:auto;...",
ctrl_card(
  selectInput("table_type", ...,
    choices=TABLE_CHOICES),
  uiOutput("tbl_pop_flag_ui"),
  uiOutput("tbl_trt_arms_ui"),
  uiOutput("table_specific_ui")
)),
card(gt_output("tbl_output"), full_screen=TRUE)
)
```

- A consistent 3/9 column split repeats across the Tables and Figures tabs one visual language for "configure, then view"
- Population flag, treatment variable, and treatment arms are rendered via `uiOutput()` server-generated, because their choices depend on which dataset is actually loaded
- `full_screen = TRUE` on the output card gives reviewers a one-click expand for detailed reading, a small but meaningful usability touch

status_chip() and ctrl_card() – tiny, reused UI functions

- `status_chip()` Renders a colored pill (checkmark ADSL 254 rows) showing which required datasets are loaded. Called twice once for ADSL, once for ADAE from one function definition.
- `ctrl_card()` A styled wrapper around `bslib::card()` with a fixed grey background and padding, used for every control panel in the app so the sidebar controls look visually distinct from the output cards.

datasets() – the one reactive everything depends on

```
datasets <- reactive({  
  if (input$data_source == "builtin")  
    return(BUILTIN)  
  req(input$upload_files)  
  withProgress(message = "Loading datasets...", {  
    for (i in seq_len(nrow(input$upload_files))) {  
      nm <- toupper(file_path_sans_ext(...))  
      df <- switch(ext,  
        "sas7bdat"=haven::read_sas(fp), ...)  
      result[[nm]] <- df  
    }  
  })  
})
```

- This is the only reactive that reads `input$data_source` or `input$upload_files` directly every other reactive in the app depends on `datasets()`, never on the raw inputs
- `req()` means: if no files are uploaded yet, this reactive simply doesn't run no error, the UI just waits
- `withProgress()` shows a native Shiny progress bar during file parsing, important once someone uploads a large ADAE

pop_flags_available() and trt_vars_available()

```
pop_flags_available <- reactive({  
  fl <- adsl_cols()[grepl("FL$", adsl_cols())]  
  valid <- fl[sapply(fl, function(v)  
    all(adsl[[v]] %in% c("Y", "N", NA)))]  
  c("All Subjects (no filter)" = "__ALL__", valid)  
})
```

- Population flags aren't hardcoded to SAFFL any column ending in FL that only contains Y/N/NA values is auto-detected as a valid population flag
- This is what makes the app work against any uploaded ADSL, not just the specific one shipped in the demo
- Treatment variables use a similar pattern: try common names (TRT01A, ARM...) first, fall back to a regex match on ^TRT[0-9]

trt_levels_for() – a function that returns a reactive

```
trt_levels_for <- function(trt_var_input,
  pop_flag_input) {
  reactive({
    req(trt_var_input(), datasets())$ADSL)
    pop <- if (pop_flag_input() != "__ALL__")
      filter(adsl, .data[[pop_flag_input()]]=="Y")
    else adsl
    sort(unique(as.character(pop[[trt_var_input()]])
  ))
})
}

tbl_trt_levels <-
  trt_levels_for(reactive(input$tbl_trt_var), ...)
fig_trt_levels <-
  trt_levels_for(reactive(input$fig_trt_var), ...)
```

- A function that builds and returns a reactive called once for the Tables tab, once for the Figures tab, avoiding duplicated reactive logic between two nearly-identical control panels
- Taking reactivities as arguments (trt_var_input, pop_flag_input) rather than plain values keeps the returned reactive correctly wired into Shiny's dependency graph
- "Screen failure" treatment levels are explicitly filtered out, since they're not a meaningful treatment arm for TFL purposes

table_data() – the dispatch reactive

```
table_data <- reactive({  
  req(input$tbl_trt_arms, input$tbl_pop_flag,  
    datasets())$ADSL)  
  withProgress(message = "Building table...", {  
    switch(input$table_type,  
      demog = build_demog(adsl, trt, pop, trts, tot),  
      ae_socpt = { req(adae)  
        build_ae_socpt(adsl, adae, trt, pop, trts, tot,  
          min_pct = input$min_pct %||% 0, sevs =  
            input$sev_filter) },  
      ...)  
    })  
  })
```

- This is the "switchboard" the only place in the server that decides which build_*() function to call, based on the dropdown selection
- req(adae) is nested inside the AE-specific branches only a demographics table doesn't need ADAE to exist, so it isn't blocked by a missing upload
- The custom %||% operator (defined at the bottom of the file) supplies a default when a control hasn't rendered yet, avoiding a NULL error on first load

render_gt() and the three table download handlers

```
output$tbl_output bold(part="header") |>  
  autofit()  
doc body_add_flextable(ft)  
print(doc, target = f)  
})
```

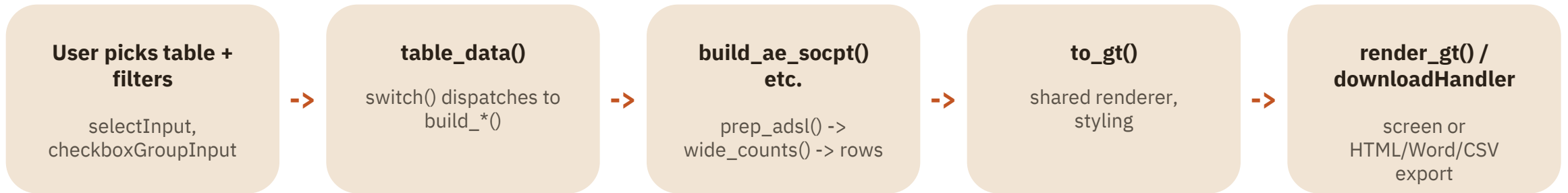
- render_gt() and every downloadHandler() independently call table_data() Shiny's reactive caching means the underlying computation runs once and is reused, not recomputed per output
- HTML export uses gtsave(), CSV export uses write.csv() on the raw data frame, and Word export rebuilds the table in flextable three renderers, one shared source of truth
- The Word export is fully programmatic: officer::read_docx() creates the document in memory, no Word template file is required

active_plot() and the PNG/SVG download handlers

```
active_plot <- reactive({
  switch(input$figure_type,
    boxplot = build_boxplot(adsl, trt, pop, trts,
      yvar=input$box_yvar, show_pts=input$box_pts,
      ...),
    ae_bar = { req(adae); build_ae_bar(adsl, adae,
      trt, pop, trts,
      top_n=input$ae_bar_topn) })
  })
output$fig_output <- renderPlot(active_plot(),
  res = 110)
output$dl_fig_png <- downloadHandler(...,
  content = function(f)
    ggsave(f, active_plot(), width=10, height=6.5,
    dpi=300))
```

- Mirrors table_data() exactly same dispatch-by-switch() pattern, applied to figures instead of tables, for a consistent codebase
- renderPlot(..., res = 110) sets screen resolution for on-screen display; ggsave(..., dpi = 300) uses a separate, higher, publication-quality resolution for the actual export the same plot object serves both
- SVG export uses the same call with device = "svg", giving reviewers a vector format for embedding in slides without quality loss

End-to-end: one click in the Tables tab

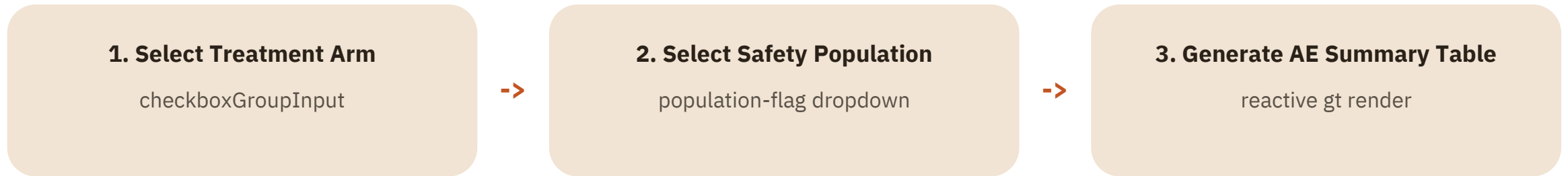


SESSION 4 • HANDS-ON BUILD

Mini TFL Builder, Interactivity, and Output Exports

Hands-on development of a mini TFL builder in R Shiny, with the audience building alongside.

Dashboard layout, navigation, and AE summary table generation



Adding interactivity for reviewing and exploring outputs

AE incidence visualization

- Bar charts ranked by incidence, matching the `build_ae_bar()` pattern from the code walkthrough
- Treatment comparison via grouped/dodged bars, same `TRT_PALETTE` as every table
- Dynamic figure updates as the top-N slider or treatment arm selection changes

Data filtering & reactive expressions

- Practice writing a `reactive()` that depends on 2-3 inputs at once
- See what happens when a dependency is missing and how `req()` handles it gracefully
- Discuss why computation lives in the reactive, not inside `render*()`, for reuse across screen and export

Exporting outputs to PDF and Word at a high level

Download/export functionality

- Wiring a `downloadHandler()` to an existing reactive the pattern is identical whether the target format is HTML, DOCX, PNG, or SVG.

Reusability considerations

- Because export handlers call the same reactive as the on-screen render, adding a new export format never means re-deriving the underlying data.

Automation considerations

- The same `build_*`() functions could run outside Shiny entirely e.g. batch-rendered nightly since they take plain data frames, not reactive objects.

SESSION 5 · AI & LLM INTEGRATION

AI and LLM Integration in R Shiny and TFL Apps

The next evolution of the TFL Builder: describe the output you need in plain language, and let an LLM route the request to the same validated functions.

High-level AI/LLM integration patterns for R Shiny (with Ollama or OpenAI API)

Pattern A – Intent router (chosen)

- LLM maps a natural-language request to a validated function call and its arguments. No LLM-generated code ever executes.

Pattern B – Narrative generator

- LLM summarizes an already-computed table or figure into plain-language commentary for a reviewer output, not computation.

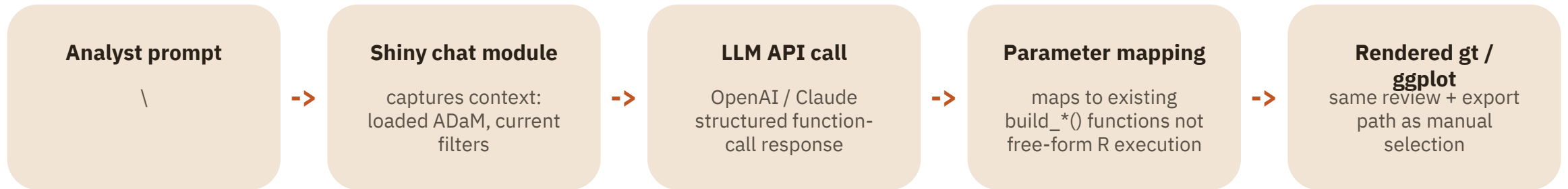
Pattern C – Local model via Ollama

- Runs entirely on-prem for compliance-sensitive settings same intent-router pattern, no data leaves the network.

Why Pattern A: LLM as intent router, not code generator

- The LLM never writes or executes arbitrary R. Instead, its job is narrow: read a request, and return a structured decision which existing, validated `build_*`() function to call and with what arguments. This is the same "function calling" capability both OpenAI's and Anthropic's Claude APIs expose natively.
- structured outputfunction callingJSON schemadeterministic execution

From natural-language request to rendered TFL



Using a chatbot interface in R Shiny

- A chat panel added alongside the existing Tables/Figures tabs. Internally it's just one more Shiny module: a scrollable message history built from a `reactiveVal()` list, a `textAreaInput` for the next message, and an `observeEvent()` on the send button that calls the LLM API and appends the response.
- Message history stored client-session-side, never persisted beyond the session by default Each response can render either plain text or trigger a table/figure update in the adjoining tab A visible "thinking / calling API..." state uses the same with `Progress()` pattern already used for data loading

Ollama Chat with R Code Execution

Ollama Model Name:

llama3.2

Enter your prompt:

Write R-code for generating a scatter plot between any two variables for iris dataset and show the data points with color coded by species

Generate & Run

Reset R session

Resetting clears variables carried over between prompts.

Chat

Plots

Prompt: Write R-code for generating a scatter plot between any two variables for iris dataset and show the data points with color coded by species

AI response:

Here's an example of how you can create a scatter plot using the iris dataset in R, where the x-axis represents

```
```r
Load required libraries
library(ggplot2)

Load the iris dataset
data(iris)

Create a new column to represent species
iris$species <- factor(iris$Species)

Scatter plot of Sepal.Length vs Petal.Length, with color coded by species
ggplot(iris, aes(x = Sepal.Length, y = Petal.Length, color = Species)) +
 geom_point() +
 labs(title = "Scatter Plot of Sepal Length vs Petal Length", x = "Sepal Length", y = "Petal Length") +
 theme_classic()
```
```

In this code:

1. We load the `ggplot2` library, which provides a powerful data visualization system in R.
2. We load the iris dataset using the built-in `data()` function.
3. We create a new column called `species` to represent the species of each flower, and we convert it to a factor.
4. We create a scatter plot using `ggplot()`, specifying the variables for x (Sepal.Length) and y (Petal.Length).

Ollama Chat with R Code Execution

Ollama Model Name:

llama3.2

Enter your prompt:

Write R-code for generating a scatter plot between any two variables for iris dataset and show the data points with color coded by species

Generate & Run

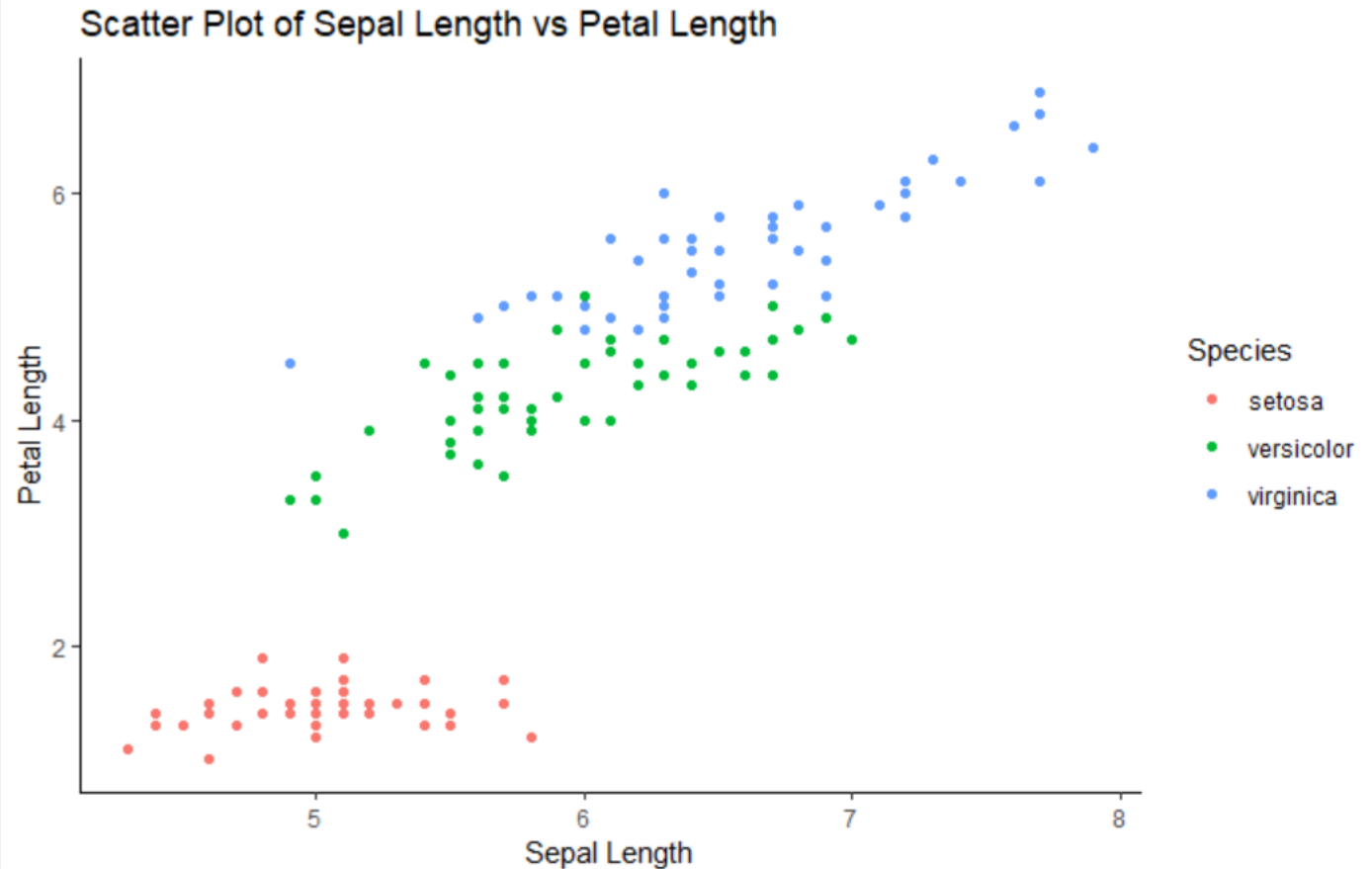
Reset R session

Resetting clears variables carried over between prompts.

Chat

Plots

From prompt: Write R-code for generating a scatter plot between any two variables for iris dataset and show the data points with color coded by species



Chat module – the shape of the server-side call

```
observeEvent(input$send, {  
  req(input$chat_msg)  
  reply <- call_llm(  
    system = SYSTEM_PROMPT,  
    tools = TFL_FUNCTION_SCHEMA,  
    user = input$chat_msg,  
    context = list(datasets = names(datasets()),  
    trt_arms = tbl_trt_levels())  
  )  
  if (reply$type == "function_call")  
    do.call(reply$fn, reply$args) # e.g.  
    build_ae_socpt(...)  
  })
```

- tools = TFL_FUNCTION_SCHEMA is where the guardrail lives the LLM is only offered the app's existing build_*() function signatures as callable "tools", nothing more general
- context passes the currently loaded dataset names and available treatment arms, so the model can't propose an arm that doesn't exist in the loaded data
- do.call(reply\$fn, reply\$args) is the only place a model's output touches execution and reply\$fn is validated against an allow-list before this line runs

Prompt design: system prompt & function schema

System prompt covers

- The app's domain: clinical TFLs, ADaM structure, population flags, treatment arms
- An explicit instruction to always resolve ambiguity by asking a follow-up, never guessing a filter
- The exact list of callable functions and what each argument means

Function schema mirrors app.R

- One schema entry per `build_*`() function, argument names matching exactly
- Enumerated values for population flag and severity, sourced live from `pop_flags_available()`
- Schema is regenerated whenever the app's own function set changes single source of truth

Why we don't let the LLM write or execute R code directly

- Determinism the same intent always maps to the same validated function, so results stay reproducible run to run
- Auditability every AI-assisted output can be traced to a specific function call and argument set, same as a manual selection
- Safety no arbitrary code execution surface inside a clinical application, regardless of what a prompt injection might attempt
- Validation reuse the `build_*`() functions are the same ones already reviewed in Sessions 3-4; the AI layer adds a front door, not a new computation path

Illustrative use case with the mini TFL app

1 • User prompt intent

- \

2 • Intent function call

- {fn: \

3 • Function call gt render

- The exact same `build_ae_socpt()` and `to_gt()` from the code walkthrough render the result no new rendering path.

Handling ambiguity and follow-up questions

- If a request under-specifies a required argument e.g. no population flag given the system prompt instructs the model to ask a clarifying question rather than default silently. This mirrors how a programmer would handle an ambiguous ad-hoc request from a study team: clarify before producing a number that could be misread.

Sample conversation transcript

- Analyst: Show me the AE summary by SOC and PT for the safety population, severe events only, minimum 5% incidence.
- Assistant: Building AE by SOC/PT Safety Analysis Set (SAFFL), Severe events, $\geq 5\%$ incidence in any arm. [table renders in the Tables tab]
- Analyst: Now compare that to Moderate and Severe combined.
- Assistant: Updating severity filter to Moderate + Severe, same population and threshold. [table re-renders]

OpenAI API – what it brings

Strengths

- Mature function-calling / structured-output support, widely documented
- Broad ecosystem familiarity for engineering teams already using it elsewhere
- Competitive latency for short, structured intent-parsing calls

Considerations

- Confirm the specific data-use and retention terms for the account tier in use
- Cost scales with context size keep only schema + metadata in the prompt, never subject-level data
- Model version pinning matters for a validated deployment track it like a package version

Claude API – what it brings

Strengths

- Strong structured-output and tool-use behavior, well suited to the intent-router pattern
- Large context window headroom if richer schema/context needs to be passed
- Anthropic's enterprise terms are a natural fit for teams already governed by them

Considerations

- Same governance homework as any vendor API confirm data residency and retention terms
- Latency and cost should be benchmarked against the actual function-schema size used here
- Model version pinning applies here too, for the same validation reasons

Ollama – the on-prem fallback

- Ollama runs an open-weight model locally or on internal infrastructure, using the same intent-router pattern a request comes in, the model returns a structured function call, app.R's existing functions execute it. No data, not even metadata, leaves the network. The trade-off is model capability: open-weight models may need more careful prompt engineering to match hosted-API reliability on structured output.

Clinical data and LLM APIs – what has to be true first

- Only metadata column names, population flags, arm labels needs to reach the LLM API for intent parsing; subject-level PHI/PII does not need to leave the app
- Redact or aggregate before any data value (vs. schema) is ever included in a prompt
- Confirm the chosen API vendor's data-use and retention terms explicitly permit this use case before go-live
- An on-prem model via Ollama removes this question entirely for the most compliance-sensitive deployments

What's next after this workshop

- Formalize the function schema for all 5 table types and 3 figure types shown today
- Prototype the chat module against a sandboxed, non-PHI dataset first
- Pilot with one study team on ad-hoc review requests before wider rollout
- Evaluate Ollama-hosted models for teams requiring fully on-prem processing

ADaM variables this app relies on – ADSL

Subject-level

- USUBJID unique subject identifier, the join key to ADAE
- SAFFL and other *FL flags auto-detected population flags
- TRT01A / TRT01P / ARM candidate treatment variables
- AGE, SEX, RACE, EOSSTT demographics & disposition inputs

ADaM variables this app relies on – ADAE

Event-level

- TRTEMFL treatment-emergent flag, filters to on-treatment events
- AEBODSYS / AEDECOD System Organ Class / Preferred Term
- AESEV severity (Mild/Moderate/Severe)
- AESER, AESDTH, AEACN seriousness, fatality, and action-taken flags for the Overview table

Data & filtering functions

- `prep_adsl()` apply population + treatment filters, compute N, synthesize Total
- `wide_counts()` count zero-fill format pivot to wide, reused 4x
- `hdr_row()` build the "(N=xx)" header row
- `fmt_n_pct()`, `fmt_mean_sd()`, `fmt_med_rng()` display formatters

Table, figure & rendering functions

- `build_demog()`, `build_dispos()`, `build_ae_overview()`, `build_ae_socpt()`, `build_ae_sev()` the 5 table builders
- `to_gt()` the single shared gt renderer for all 5 tables
- `build_boxplot()`, `build_histogram()`, `build_ae_bar()` the 3 figure builders, all styled with `tfl_theme`
- `status_chip()`, `ctrl_card()` small reusable UI components

Testing & validation considerations

Unit-testable core

- Every `build_*`() function takes plain data frames and returns a plain data frame testable with `testthat`, independent of a running Shiny session.

Golden-output comparisons

- Snapshot a known-good table/figure per builder and diff future runs against it, the same discipline as validating a SAS macro.

UI-level checks

- `shinytest2` can drive the running app end-to-end select filters, confirm the rendered table matches expectation.

Common pitfalls when building this kind of app

- Filtering logic duplicated per table instead of centralized in one `prep_adsl()`-style function the first thing that drifts out of sync
- Hardcoding a population flag or treatment variable name instead of detecting it from the loaded data
- A separate, hand-written code path for export that silently diverges from the on-screen render
- No explicit denominator (N) shown next to percentages impossible for a reviewer to sanity-check

Glossary of acronyms used today

- TFL Tables, Figures & ListingsADaM Analysis Data ModelSOC / PT System
Organ Class / Preferred TermTEAE Treatment-Emergent Adverse Event
- SAF Safety Analysis SetDSMB Data Safety Monitoring BoardCRA / DM Clinical
Research Associate / Data ManagementPHI/PII Protected/Personally
Identifiable Information

A note on 21 CFR Part 11 & validation posture

- Today's demo app is a teaching tool, not a validated deliverable. Moving a pattern like this into a submission-adjacent workflow means the same rigor applied to any statistical programming: documented requirements, IQ/OQ/PQ-style testing of each `build_*`() function, access controls on who can deploy changes, and an audit trail for what filters produced a given export.

Resources & further reading

- mastering-shiny.org
- rstudio.github.io/bslib
- rstudio.com
- pharmaverse.org
- ggplot2-book.org
- CDISC ADaM IG
- All example code in this deck is drawn directly from today's app. R copies are available from the workshop organizers on request.

End

CLOSING

Closing Session: Workshop Summary

What to take back to your team, and an open floor for questions.

Three ideas worth remembering

01 • One reactive spine

- Filter once population, treatment, arms and every table and figure inherits it consistently.

02 • Shared renderers

- One `to_gt()`, one `tfl_theme`, one export path per format less drift, easier QC.

03 • AI as a front door

- LLMs route requests to validated functions they don't replace the statistical logic underneath.

17:15 PM 17:30 PM• All

- Open floor happy to dig into any part of the app.R code, the LLM integration plan, or general R Shiny architecture questions.

THANK YOU

Let's build clinical dashboards

Mahesh Divakaran

Senior Tech Lead - R Developer, Efficacy Life Science Analytics

Venkatesh Krishnan

Senior Advisor – Computation Statistics, Eli Lilly & Company - India

IASCT - Building Interactive Dashboards for Smarter Insights Using R Shiny - 12 September 2026

Thank You